URBANVERSE

MEMBER 4 | FLOOD SIMULATION + RISK LOGIC

Project: City-Scale Digital Twin for Multi-Hazard Prediction and Urban Resilience Planning

Hackathon target: Build a convincing, working flood-risk MVP for a small pilot area in Mangalore. Use real map/geographical data where practical and clearly label simulated rainfall and risk outputs.

Your mission

Create a simple, explainable flood-risk simulation that responds to rainfall input and returns zone-level results.

Your tasks

- Work with Member 3 to understand the available zones and any elevation/water-body data.
- Research simple factors that can influence surface-flood risk, such as rainfall intensity, low elevation, proximity to water and drainage limitations where data exists.
- Design a transparent rule-based scoring method before attempting machine learning.
- Implement a function that accepts rainfall_mm and zone data and returns a score from 0 to 100 for each zone.
- Assign exactly Low, Medium or High risk labels using documented thresholds.
- Return affected zone IDs and practical, cautious response suggestions.
- Provide test scenarios and expected outputs to Member 1.

What you must hand over

- simulation/risk_model.py (or an agreed module).
- docs/risk-method.md explaining factors, assumptions and score thresholds.
- A function with a stable input/output interface.
- At least three test cases: low, moderate and heavy simulated rainfall.
- Example JSON output matching the shared API contract.

File ownership and coordination

- Own simulation/ and docs/risk-method.md.
- Use the exact zone IDs supplied by Member 3.
- Do not claim the score is a scientifically validated flood forecast.
- Do not require machine learning or a large dataset for the first working version.

Start in this order

1. Agree on the function signature with Member 1.
2. Start with a transparent rule-based score and document its assumptions.
3. Test the function with sample zones before real GIS data is ready.
4. Send Member 1 a working function and example JSON early.
5. Tune the logic only after the complete map-to-results flow works.

Your acceptance test

For the same zone, run low, medium and high rainfall scenarios. Confirm outputs are valid, explainable, use the correct zone ID and change in a sensible direction.

Team rule

Commit working changes to your own Git branch, push them, and tell the integration lead what changed. Do not edit another member's files without agreement. If you are blocked, share the exact error and the smallest reproducible example rather than silently changing the shared interface.